

1 Introduction

Universal Asynchronous Receiver/Transmitter (UART) is a widely used serial communication protocol that enables reliable data exchange between two digital systems. Unlike synchronous communication methods, UART operates in an asynchronous manner, meaning no shared clock signal is required between the transmitter and receiver. Instead, synchronization is achieved using start and stop bits that frame each transmitted byte, allowing both devices to agree on the timing of data sampling.

In a typical UART system, data is transmitted serially over a single communication line (Tx) after being converted from parallel form by the transmitter. At the receiving end, the receiver samples the incoming serial data through the Rx line and reconstructs it back into an 8-bit parallel data format. The correctness of this process depends on both modules operating at the same baud rate, which defines the number of bits transmitted per second. In this implementation, timing is controlled using a clock divider parameter (CLKS_PER_BIT), ensuring proper bit sampling and transmission delays.

This project implements a complete UART transceiver system using Verilog HDL, consisting of separate transmitter and receiver modules integrated through a top-level design. The transmitter module sends 8-bit data frames triggered by a start signal, while the receiver detects the start bit, samples each data bit at the correct timing, and reconstructs the original byte. A testbench is used to verify functionality by connecting the transmitter output directly to the receiver input and checking correctness using simulation. Finally, the design is implemented on an FPGA board, where transmitted data is observed through output indicators such as LEDs and seven-segment displays, confirming successful hardware-level UART communication.

2 Verilog RTL Code

2.1 Transmitter Module

```

1 module tx (
2     input      clk,
3     input      i_tx_start,    // pulse high to send
4     input [7:0] i_tx_data,    // byte to transmit
5     output reg  o_tx,         // serial output line
6     output reg  o_tx_done     // pulses high when done
7 );
8
9 parameter CLKS_PER_BIT = 5208; //baudrate = 9600 bps
10
11 // States
12 parameter IDLE  = 2'b00;
13 parameter START = 2'b01;
14 parameter DATA = 2'b10;
15 parameter STOP  = 2'b11;
16
17 reg [1:0] state = IDLE;
18 reg [12:0] clk_count = 0; // counts up to CLKS_PER_BIT
19 reg [2:0] bit_index = 0; // counts 0 to 7
20
21 always @(posedge clk) begin
22     case (state)
23
24         IDLE: begin
25             o_tx_done <= 1'b0;
26             o_tx      <= 1'b1; // line held high in idle
27             if (i_tx_start) begin
28                 state <= START;
29             end
30         end
31
32         START: begin
33             o_tx <= 1'b0; // pull line low
34             if (clk_count < CLKS_PER_BIT) begin

```

```

35         clk_count <= clk_count + 1;
36     end else begin
37         clk_count <= 0;
38         state      <= DATA;
39     end
40 end
41
42 DATA: begin
43     o_tx <= i_tx_data[bit_index];
44
45     if (clk_count < CLKS_PER_BIT) begin
46         clk_count <= clk_count + 1;
47     end else begin
48         clk_count <= 0;
49
50         if (bit_index == 7) begin
51             bit_index <= 0;
52             state      <= STOP;
53         end else begin
54             bit_index <= bit_index + 1;
55         end
56     end
57 end
58
59 STOP: begin
60     o_tx <= 1'b1;    // pull line high
61     if (clk_count < CLKS_PER_BIT) begin
62         clk_count <= clk_count + 1;
63     end else begin
64         o_tx_done <= 1'b1;
65         clk_count <= 0;
66         state      <= IDLE;
67     end
68 end
69
70 endcase
71 end
72
73 endmodule

```

Listing 1: UART Transmitter Module

2.2 Receiver Module

```

1 module rx (
2     input      clk,
3     input      i_rx,          // serial input line
4     output [7:0] o_rx_data,    // byte received
5     output reg  o_rx_done     // pulses high when byte is done
6 );
7
8 parameter CLKS_PER_BIT = 5208;
9
10 // States
11 parameter IDLE  = 2'b00;
12 parameter START = 2'b01;
13 parameter DATA = 2'b10;
14 parameter STOP  = 2'b11;
15
16 reg [7:0] rx_data = 0;    // internal shift register
17 reg [1:0] state    = IDLE;
18 reg [12:0] clk_count = 0; // counts up to CLKS_PER_BIT

```

```

19 reg [2:0] bit_index = 0;    // counts 0 to 7
20
21 always @(posedge clk) begin
22     case (state)
23
24         IDLE: begin
25             o_rx_done <= 1'b0;
26             if (i_rx == 1'b0) begin
27                 state <= START;
28             end
29         end
30
31         START: begin
32             if (clk_count < CLKS_PER_BIT/2) begin
33                 clk_count <= clk_count + 1;
34             end else begin
35                 if (i_rx == 1'b0) begin
36                     state <= DATA;
37                     clk_count <= 1'b0;
38                 end else begin
39                     state <= IDLE;
40                     clk_count <= 0;
41                 end
42             end
43         end
44
45         DATA: begin
46             if (clk_count < CLKS_PER_BIT - 1) begin
47                 clk_count <= clk_count + 1;
48             end else begin
49                 clk_count <= 0;
50                 rx_data[bit_index] <= i_rx;
51
52                 if (bit_index == 7) begin
53                     bit_index <= 0;
54                     state <= STOP;
55                 end else begin
56                     bit_index <= bit_index + 1;
57                 end
58             end
59         end
60
61         STOP: begin
62             if (clk_count < CLKS_PER_BIT) begin
63                 clk_count <= clk_count + 1;
64             end else begin
65                 o_rx_done <= 1'b1;
66                 clk_count <= 0;
67                 state <= IDLE;
68             end
69         end
70     endcase
71 end
72
73 assign o_rx_data = rx_data;
74
75 endmodule
76

```

Listing 2: UART Receiver Module

2.3 Top Module

```

1  module uart_top_2 (
2      input      clk,          // 50MHz onboard clock
3      input      i_rx,         // UART input from GPIO pin
4      output     o_tx,         // UART output to GPIO pin
5      input      i_tx_start,
6      output [7:0] rx_LED,
7      input [3:0] sw,
8      output [6:0] hex0,
9      output [6:0] hex1
10 );
11
12 wire [7:0] rx_data;
13 wire [7:0] tx_data = 8'b10101010;
14
15 wire [3:0] lower_nibble = rx_data[3:0];
16 wire [3:0] upper_nibble = rx_data[7:4];
17
18 //Loop through
19 reg [25:0] sec_counter = 0; // 26 bits needed to count to 50,000,000
20 wire sec_tick = (sec_counter == 26'd49_999_999); //counting starts from 0 to
    49,999,999
21
22 always @(posedge clk) begin
23     if (sec_tick)
24         sec_counter <= 0;
25     else
26         sec_counter <= sec_counter + 1;
27 end
28
29 reg [7:0] tx_byte = 8'h00;
30
31 always @(posedge clk) begin
32     if (sec_tick)
33         tx_byte <= tx_byte + 1; // increments every second
34 end
35 //
36
37 // Edge detection
38 reg prev_start = 0;
39 wire tx_pulse = (~i_tx_start) & ~prev_start;
40
41 always @(posedge clk)
42     prev_start <= ~i_tx_start;
43 //
44
45 tx uart_tx_inst (
46     .clk      (clk),
47     .i_tx_start (sec_tick), //changed from ~i_tx_start
48     .i_tx_data (tx_byte),
49     .o_tx      (o_tx),
50     .o_tx_done ()
51 );
52
53 rx uart_rx_inst (
54     .clk      (clk),
55     .i_rx      (i_rx),
56     .o_rx_data (rx_data),
57     .o_rx_done ()
58 );
59
60 function [6:0] hex_to_seg;
```

```

61     input [3:0] digit;
62     begin
63         case (digit)
64             4'h0: hex_to_seg = 7'b1000000;
65             4'h1: hex_to_seg = 7'b1111001;
66             4'h2: hex_to_seg = 7'b0100100;
67             4'h3: hex_to_seg = 7'b0110000;
68             4'h4: hex_to_seg = 7'b0011001;
69             4'h5: hex_to_seg = 7'b0010010;
70             4'h6: hex_to_seg = 7'b0000010;
71             4'h7: hex_to_seg = 7'b1111000;
72             4'h8: hex_to_seg = 7'b0000000;
73             4'h9: hex_to_seg = 7'b0011000;
74             4'hA: hex_to_seg = 7'b0001000;
75             4'hB: hex_to_seg = 7'b0000011;
76             4'hC: hex_to_seg = 7'b1000110;
77             4'hD: hex_to_seg = 7'b0100001;
78             4'hE: hex_to_seg = 7'b0000110;
79             4'hF: hex_to_seg = 7'b0001110;
80             default: hex_to_seg = 7'b1111111;
81         endcase
82     end
83 endfunction
84
85 assign hex0 = ~hex_to_seg(lower_nibble);
86 assign hex1 = hex_to_seg(upper_nibble);
87
88 assign rx_LED = tx_byte;
89
90 endmodule

```

Listing 3: UART Top Module

2.4 Testbench Module

```

1  module uart_tx_rx_tb();
2
3  // Testbench drives these
4  reg clk;
5  reg i_tx_start;
6  reg [7:0] i_tx_data;
7
8  // Module outputs      testbench just reads these
9  wire o_tx ;           // serial connection between TX and RX
10 wire o_tx_done;       // TX done flag
11 wire [7:0] o_rx_data; // received byte
12 wire o_rx_done;       // RX done flag
13
14 // Instantiate transmitter
15 tx uart_tx_inst (
16     .clk(clk),
17     .i_tx_start(i_tx_start),
18     .i_tx_data(i_tx_data),
19     .o_tx(o_tx),
20     .o_tx_done(o_tx_done)
21 );
22
23 // Instantiate receiver
24 rx uart_rx_inst (
25     .clk(clk),
26     .i_rx(o_tx),
27     .o_rx_data(o_rx_data),

```

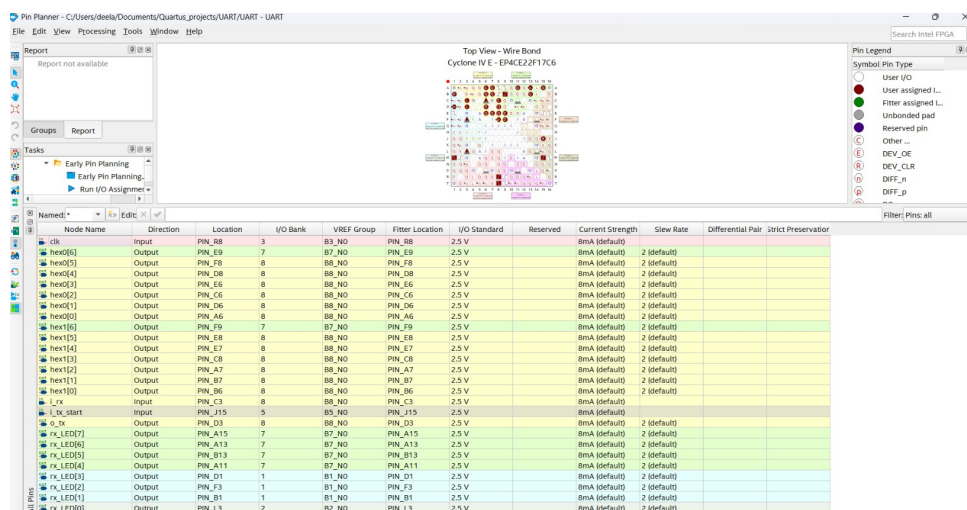
```

28     .o_rx_done(o_rx_done)
29 );
30
31 // Clock generation
32 always #10 clk = ~clk;
33
34 initial begin
35     // Initialise signals
36     clk = 0;
37     i_tx_start = 0;
38     i_tx_data = 8'h00;
39
40     // Wait a few cycles then send 0xA5
41     #100;
42     i_tx_data = 8'hA5;    //
43     #5
44     i_tx_start = 1'b1;    // trigger the transmitter
45     #20;                  // hold for one clock cycle
46     i_tx_start = 1'b0;    // release trigger
47
48     // Wait for reception to complete
49     @(posedge o_rx_done);
50
51     if (o_rx_data == 8'hA5)
52         $display("PASS: Received 0x%h", o_rx_data);
53     else
54         $display("FAIL: Received 0x%h, Expected 0xA5", o_rx_data);
55
56     $finish;
57 end
58
59 endmodule

```

Listing 4: UART Testbench Module

3 Pin Assignment



Top View - Wire Bond
Cyclone IV E - EP4K10K100-3

Node Name	Direction	Location	I/O Bank	VREF Group	Filter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Tri-state Preset
clk	Input	PN_86	3	B3_NO	PN_86	2.5 V		8mA (default)	2 (default)		
hex0[6]	Output	PN_E9	7	B7_NO	PN_E9	2.5 V		8mA (default)	2 (default)		
hex0[5]	Output	PN_F8	8	B8_NO	PN_F8	2.5 V		8mA (default)	2 (default)		
hex0[4]	Output	PN_D8	8	B8_NO	PN_D8	2.5 V		8mA (default)	2 (default)		
hex0[3]	Output	PN_E6	8	B8_NO	PN_E6	2.5 V		8mA (default)	2 (default)		
hex0[2]	Output	PN_C6	8	B8_NO	PN_C6	2.5 V		8mA (default)	2 (default)		
hex0[1]	Output	PN_D6	8	B8_NO	PN_D6	2.5 V		8mA (default)	2 (default)		
hex0[0]	Output	PN_A6	8	B8_NO	PN_A6	2.5 V		8mA (default)	2 (default)		
hex1[6]	Output	PN_F9	7	B7_NO	PN_F9	2.5 V		8mA (default)	2 (default)		
hex1[5]	Output	PN_E8	8	B8_NO	PN_E8	2.5 V		8mA (default)	2 (default)		
hex1[4]	Output	PN_E7	8	B8_NO	PN_E7	2.5 V		8mA (default)	2 (default)		
hex1[3]	Output	PN_C8	8	B8_NO	PN_C8	2.5 V		8mA (default)	2 (default)		
hex1[2]	Output	PN_A7	8	B8_NO	PN_A7	2.5 V		8mA (default)	2 (default)		
hex1[1]	Output	PN_B7	8	B8_NO	PN_B7	2.5 V		8mA (default)	2 (default)		
hex1[0]	Output	PN_B6	8	B8_NO	PN_B6	2.5 V		8mA (default)	2 (default)		
i_tx_start	Input	PN_C3	8	B8_NO	PN_C3	2.5 V		8mA (default)	2 (default)		
i_tx_data	Input	PN_J15	5	B5_NO	PN_J15	2.5 V		8mA (default)	2 (default)		
o_tx	Output	PN_D3	8	B8_NO	PN_D3	2.5 V		8mA (default)	2 (default)		
rx_LED[7]	Output	PN_A15	7	B7_NO	PN_A15	2.5 V		8mA (default)	2 (default)		
rx_LED[6]	Output	PN_A13	7	B7_NO	PN_A13	2.5 V		8mA (default)	2 (default)		
rx_LED[5]	Output	PN_B13	7	B7_NO	PN_B13	2.5 V		8mA (default)	2 (default)		
rx_LED[4]	Output	PN_A11	7	B7_NO	PN_A11	2.5 V		8mA (default)	2 (default)		
rx_LED[3]	Output	PN_D1	1	B1_NO	PN_D1	2.5 V		8mA (default)	2 (default)		
rx_LED[2]	Output	PN_F3	1	B1_NO	PN_F3	2.5 V		8mA (default)	2 (default)		
rx_LED[1]	Output	PN_B1	1	B1_NO	PN_B1	2.5 V		8mA (default)	2 (default)		
rx_LED[0]	Output	PN_L3	2	B2_NO	PN_L3	2.5 V		8mA (default)	2 (default)		

Figure 1: Pin Assignment

4 Simulation Results

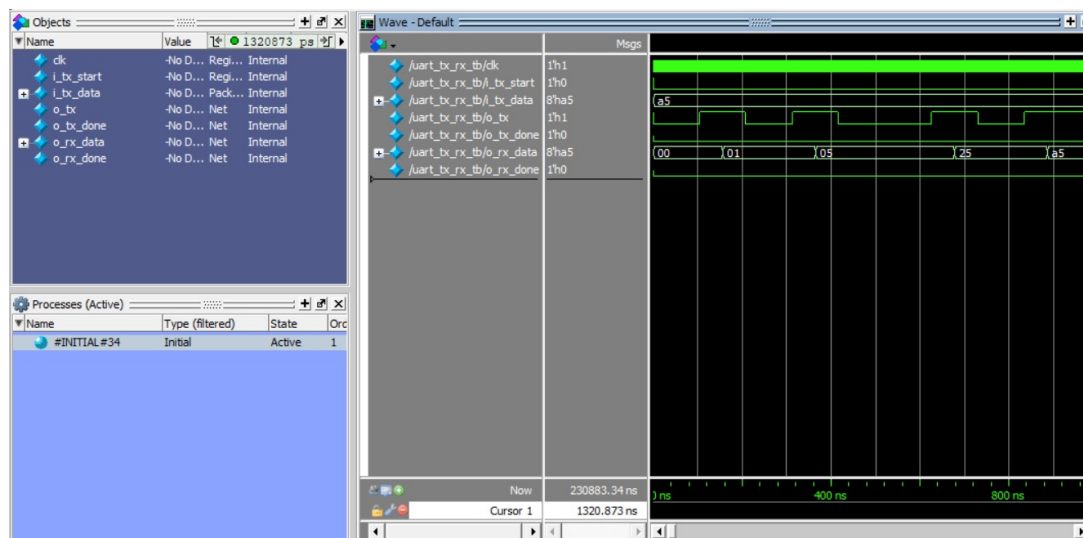


Figure 2: Waveforms in ModelSim simulation for testbench

5 FPGA Implementation

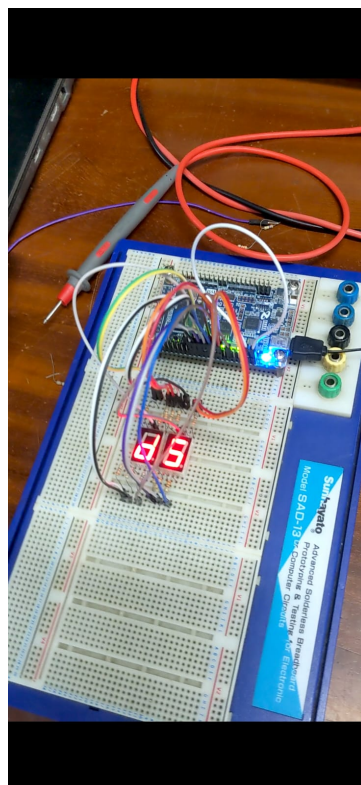


Figure 3: Transferring data between transmitter and receiver pins of same FPGA